

Assignment - 1

Name: Divyansh Bhardwaj

Registration no. - RA2211026030052

Branch - CSE AI-ML

Year/Sem/Sec - 4th / 7th / 'A'

Date - 20/09/2025

Subject - Deep Learning

1. Compare a biological neuron with the McCulloch-Pitts neuron. Highlight how the simplifications made in the mathematical model affect its learning ability.

Introduction

The field of Artificial Neural Networks is fundamentally inspired by the structure and function of the human brain. The McCulloch-Pitts (MCP) neuron, proposed in 1943, was the first significant attempt to create a mathematical model of a biological neuron. While groundbreaking, it involved several key simplifications.

Biological Neuron

A biological neuron is a complex cell responsible for transmitting information in the nervous system. Its main components are:

- **Dendrites:** Tree-like extensions that receive electrochemical signals from other neurons.
- **Soma (Cell Body):** Integrates the incoming signals. If the combined signal strength crosses a certain threshold, the neuron "fires."
- **Axon:** A long fiber that transmits the signal (an action potential) away from the cell body.
- **Synapses:** Junctions where signals are passed to other neurons. The strength of these connections (synaptic weight) can change over time, which is the basis of learning (synaptic plasticity).

McCulloch-Pitts (MCP) Neuron

The MCP neuron is a simplified mathematical construct with the following characteristics:

- It receives multiple binary inputs (0 or 1).
- It computes a simple weighted sum of these inputs. In the original model, weights were fixed (e.g., 1 for excitatory, -1 for inhibitory).
- It applies a threshold function. If the sum is greater than or equal to a predefined threshold (θ), the neuron outputs 1; otherwise, it outputs 0.

Comparison

Feature	Biological Neuron	McCulloch-Pitts (MCP) Neuron
Inputs	Complex, continuous electrochemical signals from thousands of neurons.	Simple, binary inputs (0 or 1). signals from
Weights	Synaptic strengths that are dynamic and change with experience (learning).	Fixed and manually set (e.g., 1 or -1). They are not learned.
Processing	Complex spatiotemporal summation of potentials in the soma.	Simple summation of inputs. graded
Activation	All-or-nothing "firing" (action potential) threshold.	A simple threshold logic function based on a (output is 1 if $\text{sum} \geq \theta$, else 0).
Output	A sequence of action potentials (spike train).	A single binary output (0 or 1).

Effect of Simplifications on Learning Ability

The simplifications made in the MCP model have a profound impact on its learning ability:

1. **No Learning Mechanism:** The most critical simplification is that the **weights and threshold are fixed and manually defined**. A biological neuron learns by strengthening or weakening its synaptic connections (weights). The MCP neuron lacks this ability entirely. It cannot adapt its parameters based on data or experience.
2. **Binary I/O:** The restriction to binary inputs and outputs limits its applicability to real-world problems, which often involve continuous values.
3. **Lack of Adaptability:** Because it cannot learn, an MCP neuron can only function as a pre-programmed logic gate (e.g., AND, OR). It cannot discover patterns in data on its own.

In essence, the McCulloch-Pitts neuron successfully modeled the threshold-based firing mechanism of a biological neuron but completely omitted its most crucial feature: **the ability to learn**. This simplification made it a computational curiosity rather than a practical learning tool, paving the way for later models like the Perceptron that introduced learnable weights.

2. Derive the Perceptron Learning Algorithm and show with an example how it updates weights step by step.

Derivation of the Perceptron Learning Algorithm

The Perceptron, introduced by Frank Rosenblatt, extends the MCP neuron by introducing an algorithm to learn the weights and bias from data. The goal is to find a set of weights that correctly classifies a set of linearly separable training examples.

1. Model Representation:

For an input vector $x = (x_1, x_2, \dots, x_n)$ and a weight vector $w = (w_1, w_2, \dots, w_n)$, the net input z is calculated as:

$z = w \cdot x + b = \sum(w_i * x_i) + b$ where b is the bias term.

2. Activation Function:

A step function is used to produce the output $\hat{y}$ (prediction):

$\hat{y} = 1$ if $z \geq 0$
 $\hat{y} = 0$ if $z < 0$

3. The Learning Rule:

The algorithm learns by iteratively adjusting the weights and bias whenever it makes a mistake. The update is guided by the error $(y - \hat{y})$, where y is the true label.

- **Weight Update Rule:** The change in a weight is proportional to the error and the corresponding input value.
 $w_i(\text{new}) = w_i(\text{old}) + \eta * (y - \hat{y}) * x_i$
- **Bias Update Rule:** The change in the bias is proportional to the error.
 $b(\text{new}) = b(\text{old}) + \eta * (y - \hat{y})$

Here, η (eta) is the **learning rate**, a small positive value (e.g., 0.1) that controls the step size of the update.

Step-by-Step Example: Learning the AND Function Let's train a perceptron to learn the AND function.

- **Inputs (x_1, x_2):** (0,0), (0,1), (1,0), (1,1)
- **Targets (y):** 0, 0, 0, 1

Let's initialize the parameters:

- Weights: $w_1 = 0.2, w_2 = -0.5$
- Bias: $b = 0.1$
- Learning Rate: $\eta = 0.1$

We will iterate through the data points and update the weights only on a misclassification.

Epoch 1:

1. Input: (0, 0), Target: 0

- $z = (0.2 * 0) + (-0.5 * 0) + 0.1 = 0.1$
- $\hat{y} = 1$ (since $z \geq 0$)
- **Error:** $y - \hat{y} = 0 - 1 = -1$. **Misclassified!**
- **Update weights:**
 - $w_1 = 0.2 + 0.1 * (-1) * 0 = 0.2$
 - $w_2 = -0.5 + 0.1 * (-1) * 0 = -0.5$
 - $b = 0.1 + 0.1 * (-1) = 0.0$

○

○ **New Parameters:** $w_1=0.2, w_2=-0.5, b=0.0$

2. Input: (0, 1), Target: 0

- $z = (0.2 * 0) + (-0.5 * 1) + 0.0 = -0.5$
- $\hat{y} = 0$ (since $z < 0$)
- **Error:** $y - \hat{y} = 0 - 0 = 0$. **Correctly classified.** No update.

3. Input: (1, 0), Target: 0

- $z = (0.2 * 1) + (-0.5 * 0) + 0.0 = 0.2$
- $\hat{y} = 1$ (since $z \geq 0$)
- **Error:** $y - \hat{y} = 0 - 1 = -1$. **Misclassified!**
- **Update weights:**
 - $w_1 = 0.2 + 0.1 * (-1) * 1 = 0.1$
 - $w_2 = -0.5 + 0.1 * (-1) * 0 = -0.5$
 - $b = 0.0 + 0.1 * (-1) = -0.1$

○

○ **New Parameters:** $w_1=0.1, w_2=-0.5, b=-0.1$

4. Input: (1, 1), Target: 1

- $z = (0.1 * 1) + (-0.5 * 1) - 0.1 = -0.5$
- $\hat{y} = 0$ (since $z < 0$)
- **Error:** $y - \hat{y} = 1 - 0 = 1$. **Misclassified!**
- **Update weights:**
 - $w_1 = 0.1 + 0.1 * (1) * 1 = 0.2$

$$\blacksquare w_2 = -0.5 + 0.1 * (1) * 1 = -0.4$$

$$\blacksquare b = -0.1 + 0.1 * (1) = 0.0$$

○

○ **New Parameters:** $w_1=0.2$, $w_2=-0.4$, $b=0.0$

This process continues for multiple epochs until the algorithm converges on a set of weights that correctly classifies all four data points.

3. Explain the representation power of a network of perceptrons. Why can a single perceptron not solve the XOR problem, but a multilayer perceptron can?

Representation Power of a Single Perceptron

The representation power of a single perceptron is limited to **linearly separable** functions. A perceptron works by creating a linear decision boundary (a line in 2D, a plane in 3D, or a hyperplane in higher dimensions) to separate the input space into two categories. If a set of data points can be separated by such a single hyperplane, it is called linearly separable. The logical AND and OR functions are examples of linearly separable problems.

The XOR Problem

The XOR (exclusive OR) function is the classic example of a problem that is **not linearly separable**. The truth table for XOR is:

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

If we plot these points on a graph, we can see that no single straight line can be drawn to separate the '1's from the '0's. A single perceptron, which can only create one linear boundary, will therefore always misclassify at least one point.

Representation Power of a Multilayer Perceptron (MLP)

A multilayer perceptron consists of an input layer, one or more hidden layers, and an output layer. This layered structure dramatically increases its representation power. By combining the linear boundaries of neurons in the hidden layer, an MLP can create complex, **non-linear decision boundaries**.

How an MLP Solves the XOR Problem:

An MLP with one hidden layer can solve the XOR problem. The hidden layer transforms the input data into a new representation where it becomes linearly separable. Here's how:

1. **Transform the Space:** The hidden layer can learn to represent intermediate functions. Let's say we have a hidden layer with two neurons:
 - **Neuron 1** learns the **OR** function. It creates a line that separates (0,0) from the other three points.
 - **Neuron 2** learns the **NAND** (NOT AND) function. It creates a line that separates (1,1) from the other three points.

- 2.
3. **Combine the Results:** The output layer neuron then takes the outputs of these two hidden neurons as its input. It can now learn a simple **AND** function. The final logic becomes: $XOR = (x_1 OR x_2) AND (x_1 NAND x_2)$

By doing this, the MLP creates a decision region that is bounded by two lines, successfully separating the XOR data points. The Universal Approximation Theorem states that an MLP with a single hidden layer and a non-linear activation function can approximate any continuous function to an arbitrary degree of accuracy, highlighting its immense representation power.

4. **Discuss the role of activation functions (Sigmoid, Tanh, ReLU, Leaky ReLU).**

For each, explain one practical scenario where it is most effective.

Role of Activation Functions

The primary role of an activation function in a neural network is to introduce **non-linearity**. Without a non-linear activation function, a multilayer neural network, no matter how deep, would behave like a single-layer linear model. This non-linearity allows the network to learn complex patterns and create intricate decision boundaries beyond simple straight lines.

Discussion of Specific Activation Functions:

1. Sigmoid

- **Function:** $\sigma(x) = 1 / (1 + e^{-x})$
- **Role:** It squashes any real-valued input into a range between 0 and 1.
- **Most Effective Scenario:** It is most effective in the **output layer of a binary classification problem**. The output can be directly interpreted as the probability of the input belonging to the positive class. For example, in a model that predicts whether an email is spam, a sigmoid output of 0.95 would mean a 95% probability of it being spam.

2. Tanh (Hyperbolic Tangent)

- **Function:** $\tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x})$
- **Role:** It squashes any real-valued input into a range between -1 and 1. Unlike Sigmoid, its output is zero-centered.
- **Most Effective Scenario:** It is often preferred over Sigmoid in the **hidden layers of a neural network**. Because its output is centered around zero, the gradients passed to the next layer are more balanced, which can lead to faster convergence during training.

3. ReLU (Rectified Linear Unit)

- **Function:** $f(x) = \max(0, x)$
- **Role:** It outputs the input directly if it is positive, and 0 otherwise. It is a piecewise linear function that is computationally very efficient.
- **Most Effective Scenario:** It is the most widely used and effective activation function for **hidden layers in almost all types of deep neural networks**, especially in computer vision (CNNs). Its efficiency and ability to alleviate the vanishing gradient problem have made it the default choice for modern deep learning.

4. Leaky ReLU

- **Function:** $f(x) = \max(0.01x, x)$
- **Role:** It is a variant of ReLU that allows a small, non-zero gradient (e.g., 0.01) for negative inputs.
- **Most Effective Scenario:** It is most effective in networks where the "dying ReLU" problem is a concern. This happens when a neuron's weights get updated in such a way that its input is always negative, causing it to always output 0 and stop learning. Leaky ReLU ensures that the neuron always has some gradient, preventing it from dying.

5. Using the chain rule, derive the formula for backpropagation in a 3-layer feedforward neural network and explain how partial derivatives are computed.

Introduction

Backpropagation is the cornerstone algorithm for training deep neural networks. It uses the chain rule of calculus to efficiently compute the partial derivative of the loss function with respect to every weight and bias in the network. This allows gradient descent to update the parameters in a way that minimizes the loss.

Setup for a 3-Layer Network

Let's consider a network with one input layer, one hidden layer, and one output layer.

- x : Input vector
- w_i : Weight from neuron i in the input layer to neuron j in the hidden layer.
- w : Weight from neuron j in the hidden layer to neuron k in the output layer. • z , a : Net inputs to hidden neuron j and output neuron k , respectively.
- a , a : Activations (outputs) of hidden neuron j and output neuron k . $a = \sigma(z)$. • L : Loss function, e.g., Mean Squared Error $L = \frac{1}{2} \sum (y - a)^2$.

Goal: Compute $\partial L / \partial w$ and $\partial L / \partial w_i$.

Derivation using the Chain Rule

1. Computing Partial Derivatives for Output Layer Weights (w)

We want to find how a change in w affects the total loss L . The path of influence is: $w \rightarrow z \rightarrow a \rightarrow L$.

Using the chain rule, we can write:

$$\partial L / \partial w = (\partial L / \partial a) * (\partial a / \partial z) * (\partial z / \partial w)$$

Let's break down each term:

- $\partial L / \partial a$: The derivative of the loss with respect to the output activation. For MSE, this is $-(y - a)$.
- $\partial a / \partial z$: The derivative of the activation function, $\sigma'(z)$.
- $\partial z / \partial w$: The derivative of the net input $z = \sum (w * a) + b$ with respect to w . This is simply a .

Combining these gives the final formula for the output layer:

$$\partial L / \partial w = -(y - a) * \sigma'(z) * a$$

This can be simplified by defining an "error term" $\delta = (\partial L / \partial a) * (\partial a / \partial z)$, so $\partial L / \partial w = \delta * a$.

2. Computing Partial Derivatives for Hidden Layer Weights (w_i)

Now we find how a change in w_i affects L . The path is: $w_i \rightarrow z \rightarrow a \rightarrow (\text{all } z \text{'s}) \rightarrow L$.

A change in a affects the loss through *all* the output neurons it is connected to.

Using the chain rule:

$$\partial L / \partial w_i = (\partial L / \partial a) * (\partial a / \partial z) * (\partial z / \partial w_i)$$

Let's break down these terms:

- $\partial a / \partial z$: This is simply the derivative of the hidden layer's activation function, $\sigma'(z)$.
- $\partial z / \partial w_i$: The derivative of $z = \sum(w_i * x_i) + b$ with respect to w_i . This is simply the input x_i .
- $\partial L / \partial a$: This is the crucial part. The loss L is not a direct function of a . We must sum the influence of a on the loss through all the output neurons k .

$$\partial L / \partial a = \sum (\partial L / \partial z_k) * (\partial z_k / \partial a)$$

We already know $\partial L / \partial z$ is the error term δ . The derivative $\partial z_k / \partial a$ is the weight w_{ki} . So:

$$\partial L / \partial a = \sum (\delta_k * w_{ki})$$

Combining everything for the hidden layer:

$$\partial L / \partial w_i = (\sum (\delta_k * w_{ki})) * \sigma'(z) * x_i$$

Explanation of Computation

Backpropagation is an efficient algorithm that implements this chain rule derivation. It works backward from the output layer:

1. **Forward Pass:** Compute all activations and the final loss.
2. **Backward Pass:**
 - a. **Output Layer:** Calculate the error term δ for each output neuron. Use this to compute the gradients $\partial L / \partial w$.
 - b. **Hidden Layer:** Use the error terms δ and weights w from the output layer to calculate the error term δ for each hidden neuron. Then use δ to compute the gradients $\partial L / \partial w_i$.

This process continues backward for all layers, efficiently reusing the calculated error from the subsequent layer to compute the gradients for the current layer. This avoids redundant calculations and makes training deep networks feasible.